

CustomerPilot V6

Developer Manual

Technical reference: API, DB models, state machines, cron jobs, audit actions.

For developers, not founders.

16 chapters | API-first | Prisma + Next.js 16

Table of Contents

- 1. Onboarding Engine (2-Minute Path)**
How the fast-start onboarding flow works. Default card auto-created; merchant bills immediately.
- 2. RBAC (Role-Based Access Control)**
Owner / Manager / Cashier permission matrix. Enforced server-side on every POST.
- 3. Stamp Engine**
How stamps are awarded, spent, and reversed. VIP multiplier applied at award time.
- 4. VIP Tier Engine**
Auto-upgrade customers based on lifetimeSpend. Multiplier applied at stamp award time.
- 5. Google Review Engine**
AI-drafted reviews post-redemption, bonus stamps on submit, reputation score calculation.
- 6. Birthday Engine**
7-day reminder + 7-day redemption window. Bonus stamps during window only.
- 7. Win-back Escalation Engine**
30/45/60/90 day escalation. Each stage has an owner + channel + message template.
- 8. Billing Workflow (POS)**
Bill creation -> stamp award -> WhatsApp queue. Duplicate-prevention on rapid double-click.
- 9. Customer Import Pipeline**
Multi-source import: CSV, manual, phone contacts. Duplicate prevention by phone.
- 10. Redemption Workflow**
Stock check -> manager approval branch -> stamp spend -> WhatsApp confirm.
- 11. Referral Pipeline**
Share -> friend joins -> first purchase verifies -> manager approves -> bonus stamps.
- 12. WhatsApp Message Queue**
Simulated provider with queued/sent/delivered/failed/delayed states.
- 13. Audit Log Architecture**
Append-only event log. Every monetary/permission event traceable to actor.
- 14. Subscription & Payment Gating**
Trial -> active -> payment_failed -> grace -> suspended. Write APIs gated by Merchant.status.
- 15. Cron Jobs (Background Workers)**
Daily/weekly scheduled jobs. In demo, simulated via API triggers.
- 16. State Snapshot API (Single Source of Truth)**
GET /api/state returns the entire merchant state. SPA re-fetches after every mutation.

1

Onboarding Engine (2-Minute Path)

How the fast-start onboarding flow works. Default card auto-created; merchant bills immediately.

Endpoints

POST	/api/merchants/signup	Create merchant + owner staff + trial subscription
POST	/api/onboarding	Mark an OnboardingStep as completed/skipped
GET	/api/state	Returns onboardingSteps[] for the dashboard widget

DB Models

Merchant | Staff | Subscription | StampCard | OnboardingStep

State Machine - States

signup | default_card | qr_ready | first_bill | add_staff | import_customers | add_rewards |
configure_birthday

Transitions

```
(none) -> signup // merchant.created
signup -> default_card // fastStartOnboarding() creates a StampCard from
                        DEFAULT_CARD_BY_BUSINESS[businessType]
default_card -> qr_ready // auto - POS opens
qr_ready -> first_bill // first POST /api/bills
(any) -> add_staff|import_customers // merchant completes optional step
|add_rewards|configure_birthday
```

Audit Actions

MERCHANT_CREATED | **ONBOARDING_STEP_DONE**

Notes

- fastStartOnboarding() is idempotent - safe to call on every login. It only seeds missing rows.
- OnboardingStep.skipped is a separate flag from completed. Skipped steps don't count against the progress percentage.
- Merchant.onboardingCompleted is set to true when all required steps (signup, default_card, qr_ready) are done.
- DEFAULT_CARD_BY_BUSINESS lives in src/lib/onboarding-engine.ts. Add new business types there.

2

RBAC (Role-Based Access Control)

Owner / Manager / Cashier permission matrix. Enforced server-side on every POST.

Endpoints

POST	/api/rbac/check	Pure permission check, no side effects
POST	/api/staff	Create staff (owner only)

DB Models

Staff

Audit Actions

RBAC_DENIED | STAFF_CREATED | STAFF_SUSPENDED

Notes

- can(role, permission) is the only check - call it at the top of every POST handler before any DB write.
- On denial, ALWAYS create an AuditLog with action=RBAC_DENIED. This is the insider-threat signal.
- ROLE_PERMISSIONS in src/lib/rbac.ts is the single source of truth.
- PIN is a 4-digit string. In production, hash with bcrypt.
- Cashier cannot: void/refund bills, approve referrals, create/edit rewards, manage staff, view audit log.
- Manager cannot: create/suspend staff, view subscription, change merchant settings.
- Owner: full access.

3

Stamp Engine

How stamps are awarded, spent, and reversed. VIP multiplier applied at award time.

Endpoints

POST	/api/bills	Create bill -> awardStampsForBill() called internally
POST	/api/bills/void	Reverse stamps tied to that bill
POST	/api/bills/refund	Same as void but keeps bill in ledger as 'refunded'
POST	/api/rewards/redeem	spendStampsForRedemption() consumes completed cards

DB Models

Stamp | CustomerStampCard | Bill | Redemption | Customer

State Machine - States

empty | in_progress | completed | redeemed

Transitions

(none) -> empty	// CustomerStampCard.create() on first stamp award
empty -> in_progress	// first Stamp row linked to it
in_progress -> completed	// stampsCollected >= stampsRequired
completed -> redeemed	// spendStampsForRedemption() on successful redemption
completed -> in_progress	// bill refund/void reverses stamps; completed flag cleared

Audit Actions

BILL_CREATED | BILL_VOIDED | BILL_REFUNDED | REDEMPTION_APPROVED

Notes

- stampsForAmount(rule, amount) parses rule strings: '1' = flat 1 stamp, '1 per \$N' = floor(amount/N) capped at 5.
- VIP bonus is applied AFTER stampsForAmount: actualStamps = floor(baseStamps * tier.bonusMultiplier).
- On bill void/refund, ALL Stamp rows tied to that billId are deleted. CustomerStampCard.stampsCollected is decremented.
- Edge case: if a redemption already happened on a completed card, refunding the bill that completed it leaves an 'unearned' redemption.
- Customer.lifetimeStamps and lifetimeSpend are incremented on bill creation, decremented on void/refund.

4

VIP Tier Engine

Auto-upgrade customers based on lifetimeSpend. Multiplier applied at stamp award time.

Endpoints

GET	/api/state	Returns vipTiers[] for display
-----	------------	--------------------------------

DB Models

VipTier | Customer.vipTier

State Machine - States

none | silver | gold | platinum

Transitions

none -> silver	// Customer.lifetimeSpend >= 2000
silver -> gold	// Customer.lifetimeSpend >= 5000
gold -> platinum	// Customer.lifetimeSpend >= 10000

Audit Actions

VIP_TIER_UPGRADED

Notes

- Tiers are one-way - customers never downgrade even if their spend stalls.
- DEFAULT_VIP_TIERS in src/lib/vip-engine.ts is the reference.
- applyVipBonusStamps(base, tier) = floor(base * multiplier). Floor - never inflate via rounding.
- On tier upgrade, an Achievement row is auto-created + WhatsApp queued.
- Tier check happens in awardStampsForBill() AFTER stampsForAmount() but BEFORE writing Stamp rows.

5

Google Review Engine

AI-drafted reviews post-redemption, bonus stamps on submit, reputation score calculation.

Endpoints

GET	/api/reviews	List all reviews
POST	/api/reviews/draft	Generate AI draft + queue WhatsApp to customer
POST	/api/reviews/submit	Customer submits review -> bonus stamps awarded

DB Models

Review

State Machine - States

requested | draft_ready | submitted | declined | expired

Transitions

(none) -> requested	// Redemption completed -> auto-request OR manual trigger
requested -> draft_ready	// POST /api/reviews/draft generates AI draft
draft_ready -> submitted	// Customer edits/submits via WhatsApp reply or POS
requested draft_ready -> declined	// Customer says no OR 14 days pass with no action
declined -> expired	// 30 days total elapsed

Audit Actions

REVIEW_DRAFT_GENERATED | REVIEW_SUBMITTED | REVIEW_DECLINED

Notes

- Bonus formula: base 1 stamp + 1 if 5-star + 2 if photo. Max 4 stamps per review.
- generateReviewDraft() is template-based in this demo. In production, swap for an LLM call.
- computeReputationScore() walks all submitted reviews and produces a 0-100 score.
- Photo URL is mocked in the demo. Production should accept an upload from the WhatsApp message.
- Review.requestSource values: 'post_redemption', 'post_bill', 'manual', 'birthday'.

6

Birthday Engine

7-day reminder + 7-day redemption window. Bonus stamps during window only.

Endpoints

POST	/api/birthdays	<i>Set/update customer birthday + schedule</i>
POST	/api/birthdays/redeem	<i>Redeem birthday reward (within window)</i>

DB Models

Birthday | Customer.birthday

State Machine - States

scheduled | reminder_sent | reward_ready | redeemed | missed

Transitions

(none) -> scheduled	<i>// POST /api/birthdays creates Birthday row</i>
scheduled -> reminder_sent	<i>// Cron: 7 days before birthday, send WhatsApp reminder</i>
reminder_sent -> reward_ready	<i>// Cron: birthday day arrives, window opens</i>
reward_ready -> redeemed	<i>// POST /api/birthdays/redeem during window</i>
reward_ready -> missed	<i>// Cron: 7 days after birthday, window closed, no redemption</i>

Audit Actions

BIRTHDAY_SET | BIRTHDAY_REDEEMED | BIRTHDAY_MISSED

Notes

- Customer.birthday stored as 'MM-DD' string (no year, privacy).
- Birthday.customerId is @unique - one birthday per customer.
- computeBirthdaySchedule() returns next occurrence.
- Redemption window is hard-enforced: POST /api/birthdays/redeem returns 409 if outside window.
- Birthday bonus stamps use source='birthday_bonus' on the Stamp row.
- VIP tiers can grant higher birthday bonus.

7

Win-back Escalation Engine

30/45/60/90 day escalation. Each stage has an owner + channel + message template.

Endpoints

POST	/api/winback	<i>Manually advance a customer to the next win-back stage</i>
-------------	---------------------	---

DB Models

WinBackEscalation

State Machine - States

day_30 | day_45 | day_60 | day_90 | recovered | lost

Transitions

(none)	->	day_30	// Cron: Customer.lastActiveAt is 30+ days ago
day_30	->	day_45	// Cron: 15 days pass, customer still inactive
day_45	->	day_60	// Cron: 15 more days pass
day_60	->	day_90	// Cron: 30 more days pass
day_90	->	lost	// Cron: 30 more days pass (120 total)
(any)	->	recovered	// Customer creates a bill (lastActiveAt updates)

Audit Actions

WINBACK_ESCALATED | WINBACK_RECOVERED | WINBACK_LOST

Notes

- WIN_BACK_STAGES in src/lib/winback-engine.ts is the source of truth.
- Each stage has an owner: system (auto), manager (task in queue), owner (urgent task).
- Channel: whatsapp (auto-sent), phone_call (manager/owner task to call manually).
- On customer recovery, all open WinBackEscalation rows are marked outcome='recovered'.
- At 120 days, customer.status flips to 'churned'.

8

Billing Workflow (POS)

Bill creation -> stamp award -> WhatsApp queue. Duplicate-prevention on rapid double-click.

Endpoints

POST	/api/bills	Create confirmed bill
POST	/api/bills/void	Void (clerical error) - manager+
POST	/api/bills/refund	Refund (money back) - manager+

DB Models

Bill | Stamp | CustomerStampCard | Customer | WhatsAppMessage | AuditLog

State Machine - States

draft | confirmed | voided | refunded

Transitions

(none) -> confirmed	// POST /api/bills (we don't persist draft state)
confirmed -> voided	// POST /api/bills/void (clerical error)
confirmed -> refunded	// POST /api/bills/refund (customer return)

Audit Actions

BILL_CREATED | BILL_VOIDED | BILL_REFUNDED

Notes

- Bill number format: CP-YYYY-NNNN (or SC- for Sweet Crumb). Auto-generated.
- Duplicate prevention: backend should reject a second POST /api/bills with same customerId+amount within 5 seconds.
- On bill creation: 1) Bill row, 2) awardStampsForBill(), 3) Customer stats incremented, 4) WhatsApp queued, 5) AuditLog written.
- On void/refund: stamps deleted, CustomerStampCard.stampsCollected decremented, Customer stats decremented.
- Bill.notes is overloaded: void/refund stores 'VOIDED: ' or 'REFUND: ' for audit clarity.

Customer Import Pipeline

Multi-source import: CSV, manual, phone contacts. Duplicate prevention by phone.

Endpoints

POST	/api/customers	Single customer creation
POST	/api/customers/import	Bulk CSV import with WhatsApp invite queue
POST	/api/customers/favorite	Toggle favorite flag
POST	/api/customers/notes	Update merchant notes
POST	/api/customers/block	Block/unblock customer (manager+)

DB Models

Customer

Audit Actions

CUSTOMER_CREATED | CUSTOMERS_IMPORTED | CUSTOMER_BLOCKED |
CUSTOMER_UNBLOCKED | CUSTOMER_DUPLICATE_BLOCKED

Notes

- Duplicate check: `db.customer.findFirst({ where: { merchantId, phone } })`. Returns 409 with existing customer's name.
- `referralCode` is auto-generated on creation: prefix + 8 random alphanumeric chars. @unique constraint.
- `referredById` is resolved from `referredByCode` at creation time.
- Import returns per-row results: `[{ row, status: 'imported'|'duplicate'|'invalid', customerId? }]`.
- `acquisitionChan` is auto-set: 'import' for bulk, 'referral' if `referredByCode` resolves, 'walk_in' otherwise.
- `Customer.favorite` is a merchant-set flag for fast POS access.

10

Redemption Workflow

Stock check -> manager approval branch -> stamp spend -> WhatsApp confirm.

Endpoints

POST	/api/rewards/redeem	<i>Redeem a reward for a customer</i>
POST	/api/rewards	<i>Create new reward (manager+)</i>

DB Models

Redemption | Reward | CustomerStampCard

State Machine - States

pending_approval | completed | denied | reversed

Transitions

(none) -> pending_approval	<i>// Cashier tries to redeem a requiresManagerApproval=true reward</i>
(none) -> completed	<i>// Manager/owner redeems OR cashier redeems non-approval reward</i>
pending_approval -> completed	<i>// Manager approves</i>
pending_approval -> denied	<i>// Manager denies</i>
completed -> reversed	<i>// Manager reverses (customer returned the reward)</i>

Audit Actions

REDEMPTION_PENDING_APPROVAL | REDEMPTION_APPROVED | REDEMPTION_DENIED |
REDEMPTION_OUT_OF_STOCK | REDEMPTION_REVERSED

Notes

- Stock check happens BEFORE stamp spend. If stock <= 0, return 409, no stamps consumed.
- spendStampsForRedemption finds the N oldest completed+unredeemed CustomerStampCards and marks them redeemed.
- Manager approval branch: when staff.role=CASHIER and reward.requiresManagerApproval=true, create Redemption with status='pending_approval'.
- Reward.stock is decremented atomically.
- Reward.stampsCost is in 'completed card' units. A 9-stamp card = 1 unit.

11

Referral Pipeline

Share -> friend joins -> first purchase verifies -> manager approves -> bonus stamps.

Endpoints

POST	/api/referrals	Create referral
POST	/api/referrals/approve	Approve verified referral -> bonus stamps to both
POST	/api/referrals/flag	Flag as suspected fraud (manager+)

DB Models

Referral

State Machine - States

pending | joined | verified | approved | denied | fraud_flagged

Transitions

(none) -> pending	// POST /api/referrals
pending -> joined	// Friend registers with referralCode
joined -> verified	// Friend's first bill created
verified -> approved	// Manager clicks Approve
verified -> denied	// Manager clicks Deny
(any) -> fraud_flagged	// POST /api/referrals/flag

Audit Actions

REFERRAL_CREATED | REFERRAL_APPROVED | REFERRAL_FLAGGED_FRAUD

Notes

- Referral uniqueness: db.referral.findFirst({ where: { merchantId, referrerId, friendPhone } }). 409 if duplicate.
- Approval awards 1 bonus stamp to referrer AND 1 to friend. Both get Stamp rows with source='referral_bonus'.
- Bonus is applied to the customer's active CustomerStampCard (creating one if needed).
- Fraud flag is terminal - a flagged referral can never be approved.
- referralCode is on the Customer model, not Referral.

WhatsApp Message Queue

Simulated provider with queued/sent/delivered/failed/delayed states.

Endpoints

GET	/api/state	Returns waMessages[] for the WhatsApp view
------------	-------------------	--

DB Models

WhatsAppMessage

State Machine - States

queued | sent | delivered | failed | delayed

Transitions

(none) -> queued	// Any action that creates a WhatsAppMessage
queued -> sent	// Background worker picks it up
sent -> delivered	// Provider webhook (simulated)
queued sent -> delayed	// 3 failed retry attempts
delayed -> delivered	// Provider recovers, queue flushes
delayed -> failed	// 24 hours elapsed with no delivery

Audit Actions

WHATSAPP_QUEUED | WHATSAPP_FAILED

Notes

- Templates: invite, stamp_earned, reward_ready, redemption_confirm, referral_bonus, churn_reminder_30, churn_offer_45, churn_manager_call_60, churn_owner_call_90, birthday_reminder, birthday_redeemed, review_request, review_thank_you.
- Demo mode: messages stay in 'queued' state forever. In production, a background worker flushes them.
- Customer.whatsappOptIn=false suppresses all non-essential messages.
- Photo URL on Review is a placeholder in the demo.
- Provider outage simulation: queue status='delayed' after 3 retries.

Endpoints

GET	/api/state	Returns auditLogs[] (last 200)
-----	------------	--------------------------------

DB Models

AuditLog

Audit Actions

MERCHANT_CREATED | STAFF_CREATED | BILL_CREATED | BILL_VOIDED |
REDEMPTION_APPROVED | RBAC_DENIED | REVIEW_SUBMITTED | BIRTHDAY_REDEEMED |
VIP_TIER_UPGRADED | WINBACK_ESCALATED | SUBSCRIPTION_ACTIVATED

Notes

- AuditLog is append-only - never UPDATE or DELETE a row.
- actorType: STAFF | SYSTEM | CUSTOMER. actorId + staffId populated when actorType=STAFF.
- metadata is a JSON string. Parse on read for display.
- entity + entityId let you filter by topic.
- RBAC_DENIED entries are the insider-threat signal. Owner should review weekly.
- In production, partition AuditLog by month for query performance.

Subscription & Payment Gating

Trial -> active -> payment_failed -> grace -> suspended. Write APIs gated by Merchant.status.

Endpoints

POST `/api/seed` *Reseed demo data (dev only)*

DB Models

Subscription | Merchant.status | Merchant.trialEndsAt

State Machine - States

trial | active | payment_failed | grace | suspended | cancelled

Transitions

```
(none) -> trial           // Merchant signup (14-day trial)
trial -> active           // Owner pays subscription
active -> payment_failed  // Renewal charge fails
payment_failed -> grace   // 24 hours pass with no payment update
grace -> suspended       // 7 days pass with no payment update
suspended -> active      // Owner updates payment + pays
(any) -> cancelled       // Owner cancels subscription
```

Audit Actions

SUBSCRIPTION_ACTIVATED | SUBSCRIPTION_RENEWED | SUBSCRIPTION_PAYMENT_FAILED |
SUBSCRIPTION_SUSPENDED

Notes

- Trial: 14 days. All features unlocked. No credit card required.
- Cron runs daily: if trial ended and no subscription -> mark Merchant.status='suspended' (after 7-day grace).
- Suspended state: all POST APIs return 402. GET /api/state still works (read-only). Data never deleted.
- Recovery: owner updates payment -> Subscription.status='active', Merchant.status='active'. No data loss.
- In production, integrate with Razorpay/Stripe webhooks for auto-renewal.

Notes

- DAILY 9:00 AM - Birthday reminder sweep: send WhatsApp 7 days before, mark missed if window passed.
- DAILY 9:00 AM - Win-back escalation sweep: check daysSinceLastActive, create escalations at 30/45/60/90 day thresholds.
- DAILY 10:00 AM - WhatsApp queue flush: send queued messages via provider, retry on failure (max 3).
- DAILY 6:00 PM - Daily summary WhatsApp to merchant: today's bills, revenue, new customers, top customer.
- WEEKLY Monday 9 AM - Churn risk recompute: recompute churnRiskScore for all customers.
- DAILY - Subscription check: attempt auto-renewal, on failure mark payment_failed.
- HOURLY - Review expiry sweep: mark requested reviews as 'declined' after 14 days, 'expired' after 30 days.

State Snapshot API (Single Source of Truth)

GET /api/state returns the entire merchant state. SPA re-fetches after every mutation.

Endpoints

GET**/api/state***Returns merchant + all related entities in one response*

DB Models

ALL

Notes

- Single GET returns: merchant, staff[], customers[], stampCards[], customerStampCards[], rewards[], bills[], redemptions[], referrals[], auditLogs[], waMessages[], subscriptions[], reviews[], birthdays[], vipTiers[], winBacks[], achievements[], supportTickets[], onboardingSteps[].
- Demo is single-tenant - picks first Merchant. Multi-tenant would filter by merchantId from auth context.
- All Promise.all'd for parallel DB queries.
- After any POST mutation, the frontend calls fetchState() to refresh.
- In production, replace with granular endpoints + TanStack Query for caching.
- Achievement query uses where: { customer: { merchantId } } - nested relation filter.